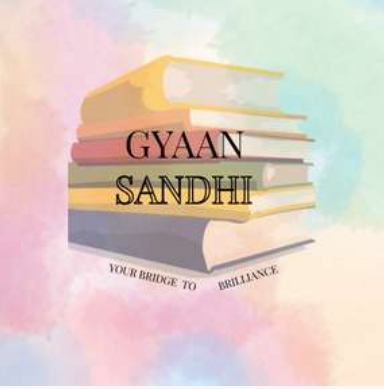




Information Security

Unit 3

**Asymmetric Key
Cryptography**

Number Theory

1. Prime Numbers

♦ Definition

A prime number is a natural number greater than 1 that has exactly two factors:

- 1
- The number itself

♦ Examples

2, 3, 5, 7, 11, 13, 17, 19, 23, ...

♦ Important Points

- 2 is the only even prime number
- 1 is not prime
- Prime numbers are the building blocks of all integers

♦ Composite Numbers

Numbers having more than 2 factors are called composite numbers

Example: 4, 6, 8, 9, 10...

♦ Fundamental Theorem of Arithmetic

Every integer greater than 1 can be written as a unique product of primes.

Example:

- $60 = 2 \times 2 \times 3 \times 5$

2. Fermat's Little Theorem

♦ Statement

If p is a prime number and a is an integer not divisible by p , then:

$$a^{p-1} \equiv 1 \pmod{p} \quad \text{OR} \quad a^p \equiv a \pmod{p}$$

If you take a number a , raise it to power $p-1$, and divide by prime p , the remainder is always 1.

♦ Example

Let $a = 2, p = 5$

$$2^4 = 16$$

$16 \div 5$ gives remainder 1

So,

$$2^4 \equiv 1 \pmod{5}$$

♦ Applications

- Cryptography (RSA algorithm)
- Modular arithmetic simplification
- Computer security systems

3. Euler's Theorem

◆ Statement

If a and n are coprime ($\gcd(a,n)=1$), then:

$$a^{\phi(n)} \equiv 1 \pmod{n}$$

Where:

- $\phi(n)$ = Euler's Totient Function (number of integers less than n that are coprime to n)

◆ What is Euler's Totient Function?

It counts numbers less than n which have no common factor with n except 1.

Example:

- $n = 10 \rightarrow$ numbers: 1,2,3,4,5,6,7,8,9
- Coprime with 10: 1,3,7,9
- So,

$$\phi(10)=4$$

◆ Example of Euler's Theorem

Let $a = 3, n = 10$

Since $\gcd(3,10)=1$:

$$3^{\phi(10)} = 3^4 = 81$$

$81 \div 10 =$ remainder 1

So:

$$3^4 \equiv 1 \pmod{10}$$

✓ Verified

◆ Special Case

If n is prime, Euler's theorem becomes **Fermat's theorem**.

Because:

$$\phi(p) = p - 1$$

Chinese Remainder Theorem (CRT)

- The Chinese Remainder Theorem (CRT) is a result in number theory that helps us solve a system of simultaneous congruences with different moduli.
- It gives a way to find a number that satisfies multiple remainders at the same time.

◆ Statement of CRT

If we have a system:

$$x \equiv a_1 \pmod{m_1}$$

$$x \equiv a_2 \pmod{m_2}$$

$$x \equiv a_3 \pmod{m_3}$$

where:

- m_1, m_2, m_3 are **pairwise coprime** (gcd of every pair is 1)

Then there exists a **unique solution modulo**:

$$M = m_1 \cdot m_2 \cdot m_3$$

Chinese Remainder Theorem (CRT)

♦ Example

Solve:

$$x \equiv 2 \pmod{3}$$

$$x \equiv 3 \pmod{5}$$

$$x \equiv 2 \pmod{7}$$

Step 1: Multiply moduli

$$M = 3 \times 5 \times 7 = 105$$

Step 2: Compute partial values

Let:

- $M_1 = 105/3 = 35$
- $M_2 = 105/5 = 21$
- $M_3 = 105/7 = 15$

Step 3: Find modular inverses

We need numbers such that:

- $35y_1 \equiv 1 \pmod{3} \Rightarrow y_1 = 2$
- $21y_2 \equiv 1 \pmod{5} \Rightarrow y_2 = 1$
- $15y_3 \equiv 1 \pmod{7} \Rightarrow y_3 = 1$

Step 4: Compute solution

$$x = (2 \cdot 35 \cdot 2) + (3 \cdot 21 \cdot 1) + (2 \cdot 15 \cdot 1)$$

$$x = 140 + 63 + 30 = 233$$

Step 5: Final answer

$$x \equiv 233 \pmod{105}$$

Now reduce:

$$233 \div 105 = \text{remainder } 23$$

✓ Final solution:

$$x = 23$$

♦ Verification

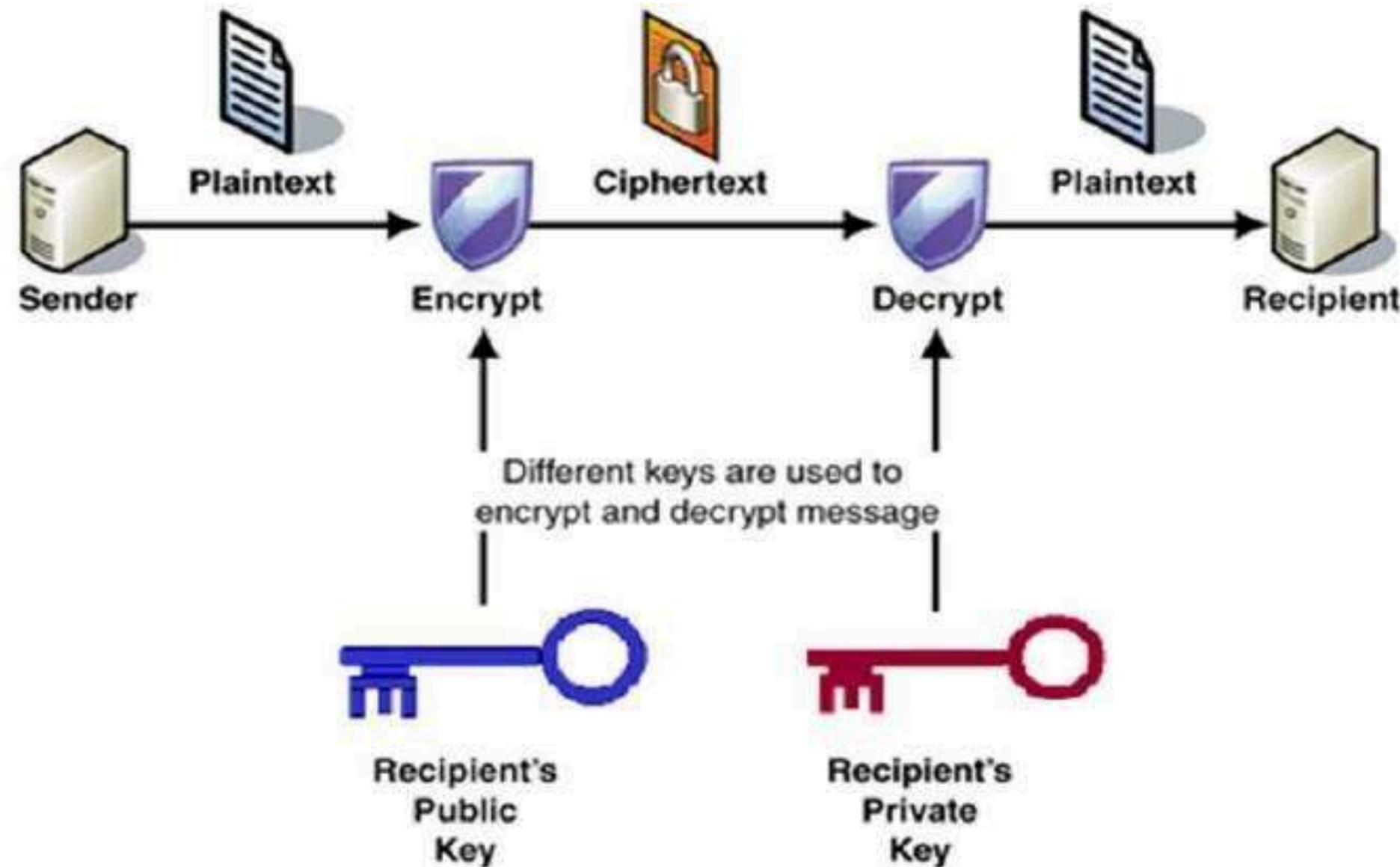
- $23 \pmod{3} = 2$ ✓
- $23 \pmod{5} = 3$ ✓
- $23 \pmod{7} = 2$ ✓

Public Key Cryptography

Public Key Cryptography (also called Asymmetric Cryptography) is a cryptographic system that uses two different keys:

- 🗝️ Public Key → used for encryption
- 🔒 Private Key → used for decryption

Both keys are mathematically related, but it is computationally infeasible to derive the private key from the public key.



Public Key Cryptography

Working of Public Key Cryptography

Step 1: Key Generation

Each user generates:

- Public Key (PK)
- Private Key (PR)

Step 2: Encryption (Sender Side)

- Sender takes receiver's public key
- Converts plaintext into ciphertext

$\text{Ciphertext} = \text{Encrypt}(\text{Public Key}, \text{Plaintext})$

Step 3: Transmission

- Encrypted message is sent over insecure network

Step 4: Decryption (Receiver Side)

- Receiver uses private key
- Converts ciphertext back to plaintext

$\text{Plaintext} = \text{Decrypt}(\text{Private Key}, \text{Ciphertext})$

Example Algorithms

Some common public key cryptographic systems:

- RSA Algorithm
- Diffie-Hellman Key Exchange (for secure key sharing)
- ECC (Elliptic Curve Cryptography)

Basis	Symmetric Key Cryptography	Asymmetric Key Cryptography
Number of keys	Uses one single key for both encryption and decryption	Uses two keys: public key and private key
Key sharing	Same key must be shared between sender and receiver	Public key can be shared openly; private key is kept secret
Security	Less secure because key must be shared	More secure because private key is never shared
Speed	Faster and requires less computation	Slower due to complex mathematical operations
Algorithm complexity	Simple algorithms	Complex algorithms
Key distribution problem	Difficult to securely share the key	Solves key distribution problem
Data usage	Suitable for large data encryption	Mostly used for small data or key exchange
Examples	AES, DES, 3DES	RSA, Diffie-Hellman, ECC
Usage	Used for bulk data encryption	Used for secure communication, digital signatures

RSA Algorithm (Rivest–Shamir–Adleman)

RSA is one of the most widely used public key cryptosystem used for secure data transmission.

It is based on the mathematical difficulty of factoring large prime numbers.

1. RSA Algorithm Steps

RSA has three main parts:

Key Generation

Encryption

Decryption

RSA ALGORITHM – WORKING EXAMPLE

1. KEY GENERATION

Step 1: Choose two prime numbers

$$p = 3, q = 11$$

Step 2: Compute $n = p \times q$

$$n = 3 \times 11 = 33$$

Step 3: Compute $\phi(n) = (p - 1)(q - 1)$

$$\phi(n) = (3 - 1)(11 - 1) = 20$$

Step 4: Choose public exponent e

$$1 < e < \phi(n) \text{ and } \gcd(e, \phi(n)) = 1$$

$$\text{Choose } e = 3$$

Step 5: Find private key d such that

$$d \times e \equiv 1 \pmod{\phi(n)}$$

$$3 \times d \equiv 1 \pmod{20} \Rightarrow d = 7$$

Final Keys

Public Key (e, n)
 $(3, 33)$

Private Key (d, n)
 $(7, 33)$

2. ENCRYPTION (SENDER SIDE)



SENDER

Plaintext (M)
 $M = 4$
(Message)

Use Receiver's
Public Key (e, n)
 $(3, 33)$

Encryption Formula
 $C = M^e \pmod n$
 $C = 4^3 \pmod{33}$
 $= 64 \pmod{33}$
 $= 31$

Ciphertext (C)
 $C = 31$



RECEIVER

----- Insecure Channel -----

3. DECRYPTION (RECEIVER SIDE)

Ciphertext (C)
 $C = 31$

Use Private Key
 (d, n)
 $(7, 33)$

Decryption Formula
 $M = C^d \pmod n$
 $M = 31^7 \pmod{33}$
 $= 4$ (Original Message)

Plaintext (M)
 $M = 4$
(Message)

SUMMARY

- Public Key $(e, n) = (3, 33)$
- Private Key $(d, n) = (7, 33)$
- Original Message $(M) = 4$
- Ciphertext $(C) = 31$
- Decrypted Message = 4

M (decrypted) = M (original)
✓ SUCCESS

HOW IT WORKS

- Sender encrypts the message using Receiver's Public Key (e, n) .
- Only the Receiver can decrypt it using his Private Key (d, n) .
- Security of RSA is based on the difficulty of factoring $n = p \times q$ (i.e., $33 = 3 \times 11$).

2. Types of Attacks on RSA

RSA is secure, but still vulnerable to certain attacks:

1. Brute Force Attack

Attacker tries all possible private keys.

- Very difficult due to large key size
- Becomes impractical for 2048-bit keys

2. Factorization Attack

Most important RSA attack.

- Attacker tries to factor n into p and q
- If p and q are found $\rightarrow$ private key is broken

3. Chosen Ciphertext Attack (CCA)

- Attacker tricks system into decrypting chosen ciphertexts
- Can reveal information about private key

4. Chosen Plaintext Attack (CPA)

- Attacker encrypts known plaintexts
- Tries to find patterns in encryption

5. Timing Attack

- Measures time taken for decryption
- Uses timing variations to guess private key

6. Man-in-the-Middle Attack

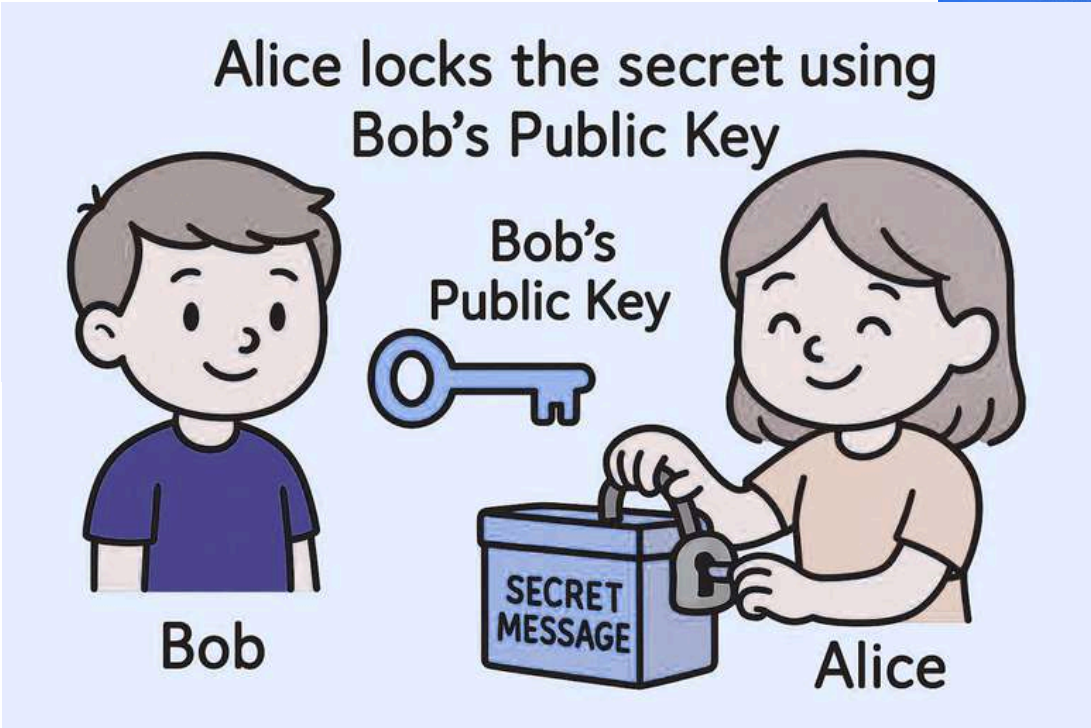
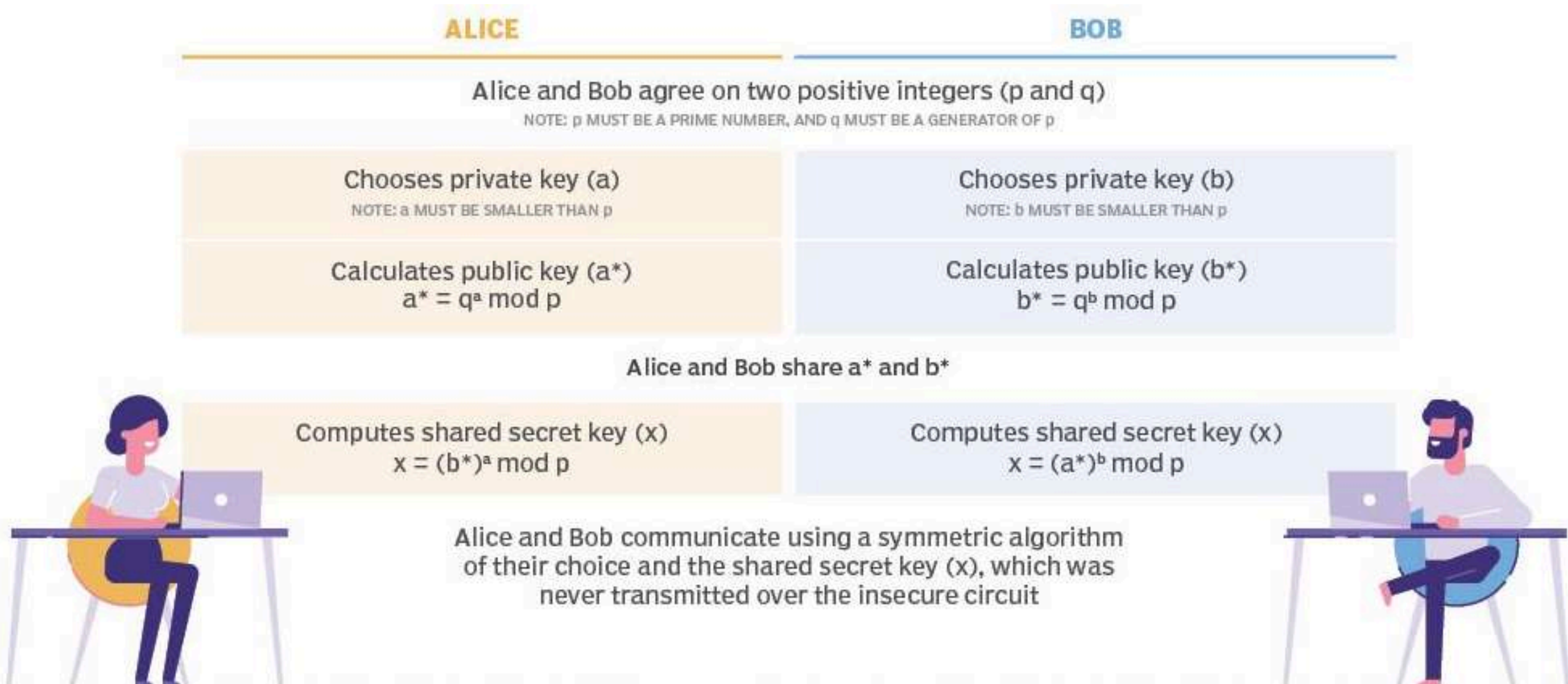
- Intercepts public key exchange
- Replaces it with attacker's key
- Leads to fake secure communication

Diffie-Hellman Key Exchange

The Diffie–Hellman Key Exchange is a method used to securely share a secret key between two parties over an insecure channel.

Important point:

It does NOT send the secret key directly, but both users independently generate the same secret key.



Let:

- $p = 23, g = 5$

Alice:

- $a = 6 \rightarrow A = 5^6 \bmod 23 = 8$

Bob:

- $b = 15 \rightarrow B = 5^{15} \bmod 23 = 19$

Shared key:

Alice:

- $S = 19^6 \bmod 23 = 2$

Bob:

- $S = 8^{15} \bmod 23 = 2$

✓ Final shared secret = 2



ElGamal Cryptographic Algorithm

The ElGamal algorithm is an asymmetric (public key) cryptography technique used for secure data transmission. It is based on the concept of discrete logarithms and is widely used for encryption and digital signatures.

1. Key Idea

- Uses two keys:
 - Public Key → used for encryption
 - Private Key → used for decryption
- Security depends on the difficulty of solving discrete logarithm problem



ELGAMAL ALGORITHM

ElGamal is an asymmetric key cryptographic algorithm based on the
Discrete Logarithm Problem.

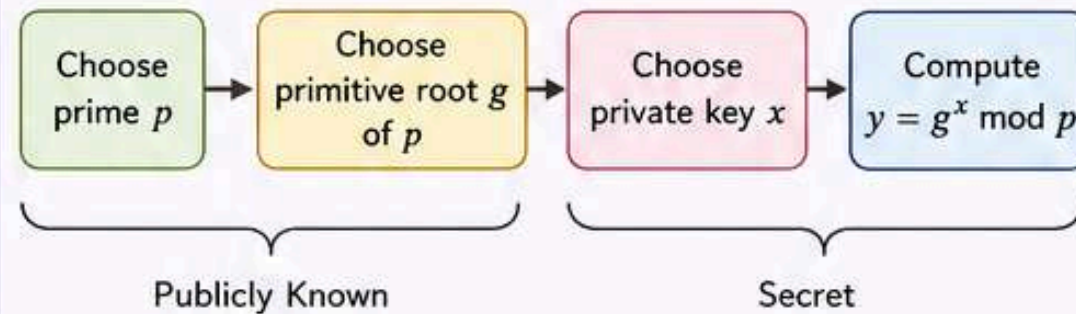
1. KEY GENERATION

1. Choose a large prime number p
2. Choose a primitive root g of p
3. Choose a random number x such that $1 < x < p - 1$ (private key)
4. Compute $y = g^x \bmod p$ (public key component)

Public Key : (p, g, y)
Private Key : x

p = large prime number
 g = primitive root of p
 x = private key ($1 < x < p - 1$)
 $y = g^x \bmod p$ (public key component)

KEY GENERATION DIAGRAM



2. ENCRYPTION

To send a message M (where $0 < M < p$):

1. Choose a random number k such that $1 < k < p - 1$
2. Compute $c_1 = g^k \bmod p$
3. Compute $c_2 = M \cdot y^k \bmod p$

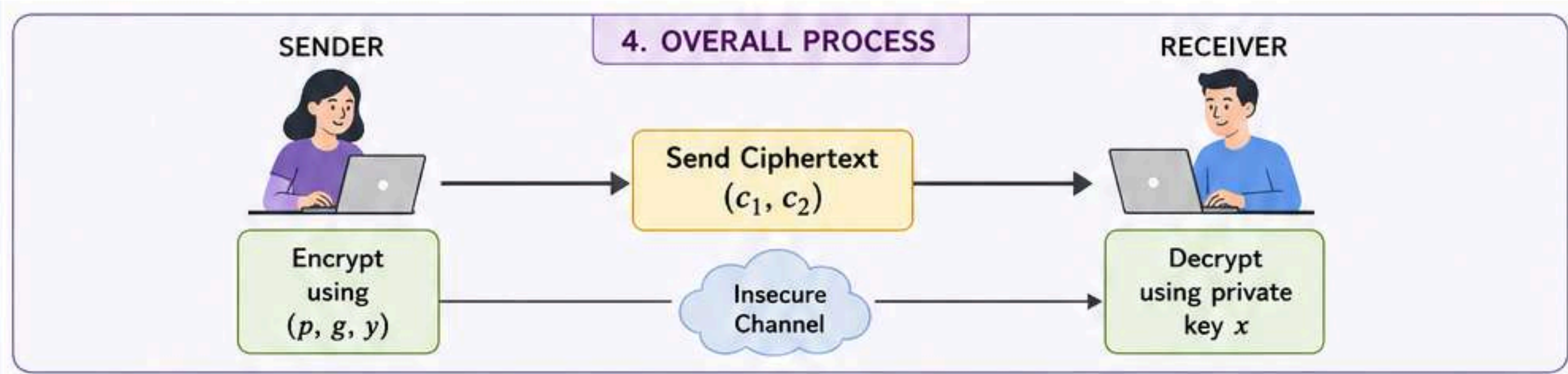
Ciphertext = (c_1, c_2)

3. DECRYPTION

To recover the message M :

1. Compute $s = c_1^x \bmod p$
2. Compute $s^{-1} \bmod p$ (modular inverse of s)
3. Recover $M = c_2 \cdot s^{-1} \bmod p$

Here, $s = (g^k)^x = g^{kx} = (g^x)^k = y^k \pmod{p}$



5. EXAMPLE (Small Numbers)

- Let $p = 23$ (prime)
- Let $g = 5$ (primitive root of 23)
- Private key $x = 6$
- Public key $y = g^x \bmod p = 5^6 \bmod 23 = 8$

Public Key = (23, 5, 8), Private Key = 6

Encryption (Let $M = 13$)

- Choose $k = 7$
- $c_1 = g^k \bmod p = 5^7 \bmod 23 = 17$
- $c_2 = M \cdot y^k \bmod p = 13 \cdot 8^7 \bmod 23 = 20$

Ciphertext = (17, 20)

Decryption

- $s = c_1^x \bmod p = 17^6 \bmod 23 = 2$
- $s^{-1} \bmod 23 = 12$ (since $2 \cdot 12 = 24 \equiv 1 \bmod 23$)
- $M = c_2 \cdot s^{-1} \bmod p = 20 \cdot 12 \bmod 23 = 13$ (original message)

6. FEATURES

- ✓ Based on Discrete Logarithm Problem
- ✓ Provides confidentiality and authentication (used for digital signatures also)
- ✓ Different ciphertext for same message (due to random k)
- ✓ Secure and widely used

7. DISADVANTAGES

- ✗ Slower than symmetric key cryptography
- ✗ Requires large key sizes for high security
- ✗ More computationally intensive

8. APPLICATIONS

- Secure communication
- Digital signatures (ElGamal Signature Scheme)
- Used in many cryptographic protocols

Elliptic Curve Cryptography

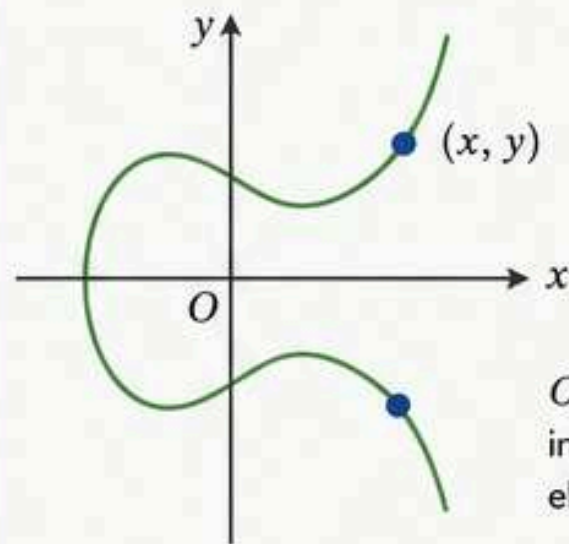
1. ELLIPTIC CURVE

An elliptic curve over a finite field F_p (p is prime) is defined by the equation:

$$y^2 \equiv x^3 + ax + b \pmod{p}$$

where,

$$4a^3 + 27b^2 \not\equiv 0 \pmod{p} \text{ (non-singular curve)}$$



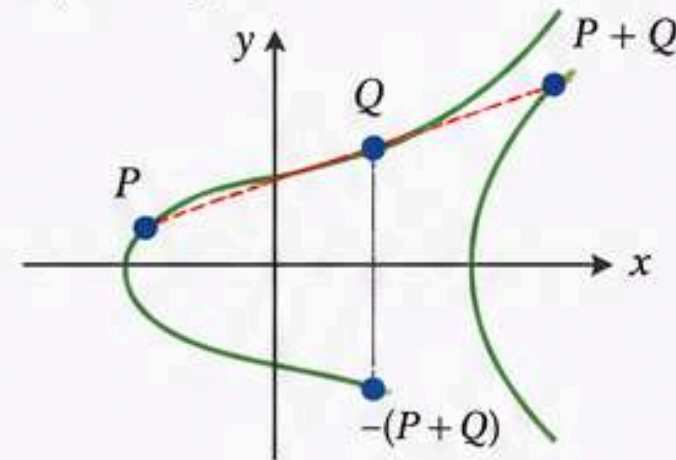
O : Point at infinity (identity element)

Set of points on the curve + O forms an abelian group.

2. BASIC OPERATIONS

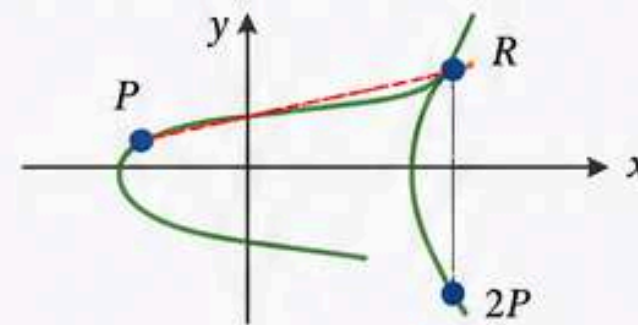
Point Addition ($P + Q$)

Draw a line through P and Q , it intersects the curve at R . Reflect R about x -axis to get $P + Q$.



Point Doubling ($2P$)

Tangent at P intersects curve at R . Reflect R about x -axis to get $2P$.



Scalar Multiplication:

$$kP = P + P + \dots + P \text{ (k times)}$$

3. KEY GENERATION

- Select elliptic curve E over F_p (domain parameters)
- Choose a base point (generator) G on E of large prime order n
- Choose private key d randomly from $[1, n - 1]$
- Compute public key $Q = dG$

Private Key : d

Public Key : $Q = dG, (E, G, n, p, a, b)$

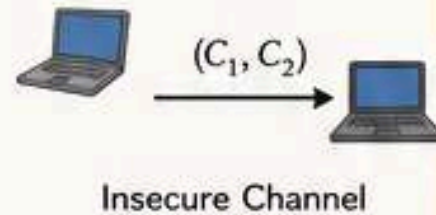
Domain Parameters

- p : prime number
- E : elliptic curve $y^2 = x^3 + ax + b \pmod{p}$
- a, b : curve coefficients
- G : base point (generator)
- n : order of G ($nG = O$)
- h : cofactor ($h = \#E(F_p) / n$)

4. ECC ENCRYPTION (ECIES Example)

Sender

- Message : M
- Convert M to point P_M on E
- Choose random integer k from $[1, n - 1]$
- Compute:
 $C_1 = kG$
 $C_2 = P_M + kQ$
- Ciphertext: (C_1, C_2)



Receiver (with private key d)

- Compute: $dC_1 = d(kG) = k(dG) = kQ$
- Compute: $P_M = C_2 - dC_1$
- Recover message M from P_M

Note: ECMQV, ECDH, ECIES are common ECC based schemes.

5. ECC DECRYPTION (ECIES)

- Compute: $dC_1 = d(kG) = k(dG) = kQ$
- Compute: $P_M = C_2 - dC_1$
- Recover message M from P_M

Point Subtraction:

$P - Q = P + (-Q)$, where $-Q$ is reflection of Q about x -axis.

6. EXAMPLE (Small Numbers)

Let the curve: $y^2 \equiv x^3 + 2x + 3 \pmod{17}$
Take base point $G = (5, 1)$ of order $n = 19$
Private key: $d = 7$
Public key: $Q = dG = 7G = (0, 6)$

Calculation	Result
$2G$	$(6, 3)$
$3G$	$(10, 6)$
$4G$	$(3, 1)$
$5G$	$(9, 16)$
$6G$	$(16, 13)$
$7G$	$(0, 6) = Q$

Encryption:

- Message point $P_M = (13, 7)$
- Choose $k = 5$
- $C_1 = kG = 5G = (9, 16)$
- $C_2 = P_M + kQ = (13, 7) + 5(0, 6) = (7, 6)$
- Ciphertext = $((9, 16), (7, 6))$

Decryption:

- $dC_1 = 7(9, 16) = 5(0, 6) = (0, 11)$
- $P_M = C_2 - dC_1 = (7, 6) - (0, 11) = (13, 7)$
- Message recovered.

7. ADVANTAGES OF ECC

- ✓ High security with small key sizes
- ✓ Faster and more efficient
- ✓ Lower power consumption (good for mobile/IoT)
- ✓ Smaller key size than RSA for same security level

8. DISADVANTAGES

- ✗ More complex mathematically
- ✗ Implementation is error-prone
- ✗ Requires proper random number generation

9. APPLICATIONS

- Secure communication (SSL/TLS)
- Digital Signatures (ECDSA)
- Key Exchange (ECDH)
- Blockchain, Cryptocurrencies
- IoT and Mobile Security

Key Insight: ECC security is based on the Elliptic Curve Discrete Logarithm Problem (ECDLP):
Given P and $Q = dP$, find d .

ECC provides the same security as RSA but with much smaller key sizes.



LIKE

SHARE

SUBSCRIBE